



FIAP · FASE 1 · GRUPO 16

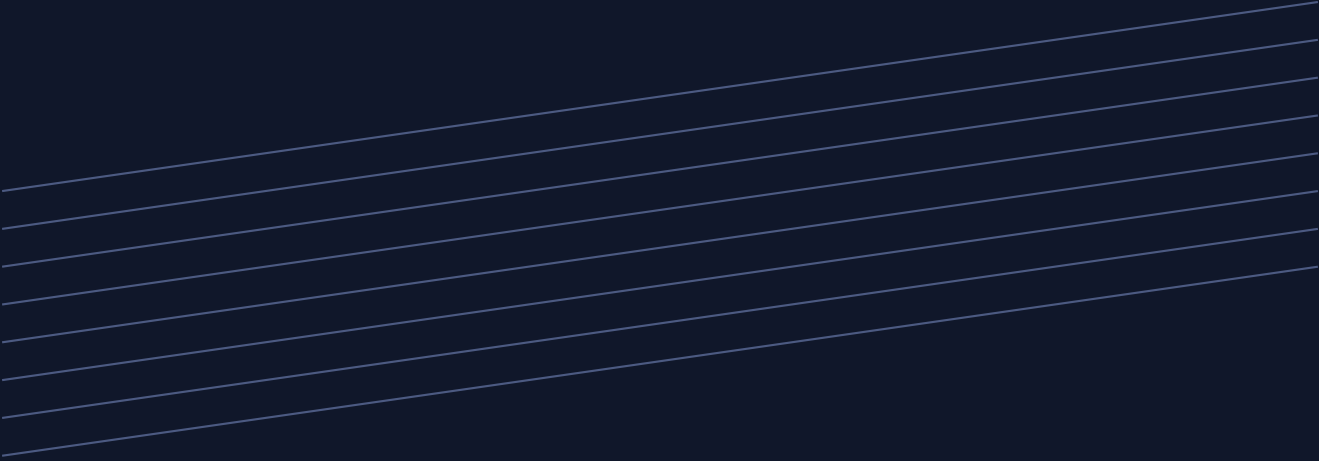
# RELATÓRIO OPERACIONAL DE PRÉ-DECOLAGEM

Atividade Integradora · Decolagem da Missão

[github.com/bkampdev/FIAP-PBL-FASE-1](https://github.com/bkampdev/FIAP-PBL-FASE-1)

Davi Coninck Cassaro · RM572772  
Guilherme Cardoso Bremenkamp · RM574229  
Lorenzo Mendes Rocha · RM575361  
Gabriel Gonzales · RM574746  
Eduardo Backes Klauck · RM575889

São Paulo · 16 de setembro de 2026



# Resumo executivo

Este trabalho implementa uma simulação educacional de pré-decolagem. O sistema lê dados sintéticos, valida o contrato de entrada, calcula o balanço energético, verifica faixas seguras e apresenta uma decisão explicável. A missão só é liberada quando todos os critérios são satisfeitos e o saldo energético é positivo.

**A análise por IA apoia a interpretação, mas não substitui as regras determinísticas nem a revisão humana. Os limites são hipóteses didáticas do grupo e não parâmetros certificados de uma nave real.**

## Entregáveis consolidados

| Item do FIAP ON      | Artefato neste repositório                  |
|----------------------|---------------------------------------------|
| 1.1 Telemetria       | docs/telemetria.md, dados/*.json e notebook |
| 1.2 Algoritmo        | docs/algoritmo.md e pseudocódigo            |
| 1.3 Script Python    | src/, notebook e testes                     |
| 1.4 Energia          | src/energia.py e docs/energia.md            |
| 1.5 Análise por IA   | docs/analise-ia.md                          |
| 1.6 Reflexão crítica | docs/reflexao_critica.md                    |
| PDF + fonte          | relatorio/relatorio-pre-decolagem.pdf e .md |
| README + prints      | README.md e evidencias/                     |

## Links públicos

[Repositório público no GitHub](#)

[Notebook pre\\_decolagem.ipynb](#)

# 1. Objetivo e arquitetura

O objetivo é responder PRONTO PARA DECOLAR ou DECOLAGEM ABORTADA com base em telemetria e energia, acumulando os motivos encontrados.

| Etapa | Responsabilidade                                    | Módulo              |
|-------|-----------------------------------------------------|---------------------|
| 1     | Ler JSON e tratar arquivo ausente/malformatado      | src/validacao.py    |
| 2     | Validar presença, tipos e domínios                  | src/validacao.py    |
| 3     | Calcular energia inicial, perdas, saldo e autonomia | src/energia.py      |
| 4     | Avaliar limites, integridade, módulos e viabilidade | src/verificacao.py  |
| 5     | Integrar o fluxo e padronizar o resultado           | src/missao.py       |
| 6     | Apresentar texto sem recalculer valores             | src/apresentacao.py |

## Princípios de projeto

Separação de responsabilidades. Validação de formato, cálculo energético, decisão e apresentação ficam em módulos diferentes.

Falha segura. Dado ausente, inválido ou resultado energético incompleto nunca libera a missão.

Explicabilidade. O verificador percorre todas as regras aplicáveis e acumula os motivos, em vez de ocultar problemas após a primeira falha.

Reprodutibilidade. Os cenários são JSONs versionados e o núcleo não exige API key ou acesso à internet.

## 2. Telemetria — requisito 1.1

O contrato distingue domínio válido de faixa operacional segura. Uma falha operacional válida deve chegar ao verificador; um valor impossível é rejeitado antes da decisão.

| Campo                  | Unidade / tipo | Domínio válido | Faixa segura |
|------------------------|----------------|----------------|--------------|
| temperatura_interna_c  | °C · número    | -100 a 150     | 15 a 30      |
| temperatura_externa_c  | °C · número    | -200 a 150     | -150 a 120   |
| integridade_estrutural | inteiro 0/1    | 0 ou 1         | 1            |
| energia_pct            | % · número     | 0 a 100        | 50 a 100     |
| pressao_tanque_kpa     | kPa · número   | > 0 e ≤ 1000   | 90 a 110     |
| modulos                | objeto         | cinco módulos  | todos em OK  |

### Módulos críticos

Suporte de vida, energia, comunicação, propulsão e navegação. Os estados aceitos são OK e FALHA.

### Cenários versionados

| Arquivo                | Alteração principal         | Resultado esperado          |
|------------------------|-----------------------------|-----------------------------|
| nominal.json           | nenhuma                     | PRONTO PARA DECOLAR         |
| falha_temperatura.json | temperatura interna = 31 °C | aborto por temperatura      |
| falha_modulo.json      | propulsão = FALHA           | aborto por módulo           |
| falha_energia.json     | consumo = 80 kWh            | aborto por energia          |
| entrada_invalida.json  | pressão ausente             | aborto por entrada inválida |

### 3. Algoritmo — requisito 1.2

As fronteiras das faixas são inclusivas. A energia só é viável com saldo estritamente positivo. Todas as falhas aplicáveis são reunidas antes da decisão.

```
INÍCIO
  carregar cenário
  validar estrutura, tipos e domínios
  SE houver erros:
    retornar DECOLAGEM ABORTADA e todos os erros
  calcular energia inicial, perdas, energia útil, saldo e autonomia
  motivos <- lista vazia
  PARA cada faixa segura:
    registrar valores abaixo ou acima dos limites
  verificar integridade estrutural
  PARA cada módulo crítico:
    registrar módulo em falha
  SE saldo energético <= 0:
    registrar energia insuficiente
  SE motivos estiver vazia:
    retornar PRONTO PARA DECOLAR
  SENÃO:
    retornar DECOLAGEM ABORTADA e todos os motivos
FIM
```

#### Percurso nominal

Temperaturas, energia e pressão estão dentro das faixas; a integridade é 1; todos os módulos estão em OK; o saldo energético é 56 kWh. Como a lista de motivos permanece vazia, o resultado é PRONTO PARA DECOLAR.

#### Percurso com falha

Com temperatura interna de 31 °C, o verificador registra que o valor excede o máximo de 30 °C. O saldo energético positivo não elimina a falha térmica. A decisão é DECOLAGEM ABORTADA e o motivo permanece visível.

#### Contrato de saída

```
{"decisao": "PRONTO PARA DECOLAR", "motivos": []}
{"decisao": "DECOLAGEM ABORTADA", "motivos": ["..."]}
```

## 4. Script Python — requisito 1.3

O notebook importa os módulos do repositório e executa cenários reais. O trecho abaixo resume a integração; cada função mantém uma responsabilidade única.

```
from src.missao import executar_cenario, LIMITES_PADRAO
from src.apresentacao import formatar_resultado

# Executar a partir da raiz do repositório.
resultado = executar_cenario("dados/nominal.json", LIMITES_PADRAO)
print(formatar_resultado(resultado))
```

### Tratamento de erros

Arquivo ausente, JSON malformatado, campo obrigatório ausente, tipo incorreto, número não finito ou percentuais fora de domínio geram aborto explicado. Nenhum dado crítico é preenchido silenciosamente com um valor seguro.

### Apresentação

A função `formatar_resultado` recebe o dicionário final e retorna uma string. Ela mostra cenário, decisão, motivos e energia, lida com valores indisponíveis e não altera o objeto recebido.

### Testes automatizados

**A suíte cobre nominal, limites, cada módulo crítico, integridade, energia positiva/nula/negativa, entradas inválidas, integração e apresentação. Na versão final, 52 testes foram aprovados.**

## 5. Análise energética — requisito 1.4

A capacidade total, a carga, as perdas e o consumo são expressos em kWh/%; a potência média é usada somente para estimar a autonomia após a decolagem.

| Etapa           | Fórmula                                             | Exemplo nominal                    |
|-----------------|-----------------------------------------------------|------------------------------------|
| Energia inicial | $\text{capacidade} \times \text{carga} / 100$       | $100 \times 80\% = 80 \text{ kWh}$ |
| Perdas          | $\text{energia inicial} \times \text{perdas} / 100$ | $80 \times 5\% = 4 \text{ kWh}$    |
| Energia útil    | $\text{energia inicial} - \text{perdas}$            | $80 - 4 = 76 \text{ kWh}$          |
| Saldo           | $\text{energia útil} - \text{consumo}$              | $76 - 20 = 56 \text{ kWh}$         |
| Autonomia       | $\text{saldo} / \text{potência, se saldo} > 0$      | $56 / 10 = 5,6 \text{ h}$          |

**Regra de viabilidade: saldo maior que zero. Com saldo zero ou negativo, viável é falso e a autonomia é None.**

### Cenário de falha energética

Com a mesma energia inicial e perdas, mas consumo de decolagem de 80 kWh, o saldo é  $76 - 80 = -4 \text{ kWh}$ . A decisão é DECOLAGEM ABORTADA e a autonomia temporal não é calculada.

### Validações numéricas

Capacidade precisa ser positiva; carga e perdas devem estar entre 0% e 100%; consumo não pode ser negativo; potência, quando informada, precisa ser positiva. NaN e infinito são rejeitados.

## 6. Análise assistida por IA — requisito 1.5

A consulta ao Codex/GPT-5 pediu classificação dos dados, anomalias e sugestões de risco para os cenários nominal, temperatura e propulsão. O prompt forneceu as faixas e fórmulas e proibiu autorização operacional real.

| Cenário     | Classificação da IA                          | Revisão humana                       |
|-------------|----------------------------------------------|--------------------------------------|
| Nominal     | sem anomalias; saldo 56 kWh; autonomia 5,6 h | confere com JSON e código            |
| Temperatura | 31 °C excede o máximo de 30 °C               | confere; energia não elimina a falha |
| Propulsão   | módulo crítico em FALHA                      | confere; exige aborto                |
| Limitação   | não há dados para causa interna da falha     | correto; não inventar diagnóstico    |

**Em caso de divergência, prevalecem os dados versionados e os módulos determinísticos de validação, energia e verificação.**

### Geração sintética opcional

O módulo `src/geracao.py` usa `GaussianMixture` como extensão para produzir dados sintéticos. Ele não participa da decisão autoritativa e não é necessário para a execução do notebook principal.

## 7. Reflexão crítica — requisito 1.6

### Ética e responsabilidade

Uma decisão de segurança precisa ser explicável. O sistema acumula motivos, informa valores e limites, e aborta quando faltam dados indispensáveis. A IA serve como apoio interpretativo; não recebe autoridade para aprovar a missão.

### Impacto social

Tecnologias espaciais podem gerar aplicações em medicina, agricultura, manufatura e observação terrestre. Ao mesmo tempo, o investimento espacial disputa recursos com outras prioridades sociais. O simulador não resolve essa decisão política, mas evita apresentar dados sintéticos como segurança real.

### Sustentabilidade tecnológica

O balanço energético torna perdas e consumo explícitos e evita declarar autonomia quando o saldo é nulo ou negativo. O projeto não mediu emissões ou consumo computacional; portanto, não faz alegações ambientais sem dados.

### Síntese

**O principal aprendizado é que uma verificação segura não se limita a responder “sim” ou “não”: ela deve explicar, registrar incertezas e permitir auditoria.**

## 8. Evidências — cenário nominal



Figura 1 — Telemetria nominal, saldo de 56 kWh, autonomia de 5,6 h e decisão PRONTO PARA DECOLAR.

A imagem foi diagramada a partir do output persistido na célula correspondente do notebook executado. O JSON de origem é dados/nominal.json.

## 9. Evidências — aborto por temperatura



Figura 2 — Temperatura interna de 31 °C, acima do limite de 30 °C, e decisão DECOLAGEM ABORTADA.

Os demais dados permanecem nominais, o que isola a causa do aborto. O JSON de origem é dados/falha\_temperatura.json.

## 10. Evidências — aborto por energia



Figura 3 — Saldo de -4 kWh, autonomia não calculada e decisão DECOLAGEM ABORTADA.

O consumo de decolagem de 80 kWh supera a energia útil de 76 kWh. O JSON de origem é dados/falha\_energia.json.

# 11. Validação, conclusão e referências

## Verificação executada em 16/09/2026

```
python -m unittest discover -s tests -v
python -m jupyter nbconvert --to notebook --execute --inplace notebooks/pre_decolagem.ipynb
```

**Resultado: 52 testes aprovados, 0 falhas, e notebook executado integralmente.**

| Cenário              | Decisão             | Saldo        | Autonomia     |
|----------------------|---------------------|--------------|---------------|
| nominal              | PRONTO PARA DECOLAR | 56 kWh       | 5,6 h         |
| falha de temperatura | DECOLAGEM ABORTADA  | 56 kWh       | 5,6 h         |
| falha de energia     | DECOLAGEM ABORTADA  | -4 kWh       | não calculada |
| entrada inválida     | DECOLAGEM ABORTADA  | indisponível | indisponível  |

## Conclusão

O projeto atende aos seis itens técnicos solicitados e inclui notebook executável, README com instruções e prints, fonte editável e PDF. A separação entre validação, energia, decisão e apresentação permite rastrear cada resultado e mantém explícito o caráter didático dos dados.

## Referências

FIAP ON. Atividade Integradora — Relatório Operacional de Pré-Decolagem. Consulta em 16 set. 2026.

NASA. Technology Transfer and Spinoffs. [nasa.gov](https://www.nasa.gov).

Projeto público: [github.com/bkampdev/FIAP-PBL-FASE-1](https://github.com/bkampdev/FIAP-PBL-FASE-1).

Base integrada: a487de4. O notebook e os artefatos foram consolidados em 16/09/2026. A presença do PDF no GitHub não equivale à submissão no FIAP ON.